

# NEGATIVE SELF-DISTILLATION: LEARNING TO REASON BY AVOIDING FLAWS

Rongcan Pei<sup>1</sup>, Zhepei Wei<sup>1</sup>, Shuyao Xu<sup>2</sup>, Xinyu Zhu<sup>1</sup>, Wei-Lin Chen<sup>1</sup>, and Yu Meng<sup>1</sup>

<sup>1</sup>Department of Computer Science, University of Virginia <sup>2</sup>Stanford University  
 {peirongcan, zhepei.wei, xinyuzhu, wlchen, yumeng5}@virginia.edu  
 shuyao@stanford.edu

 GitHub  Hugging Face

## ABSTRACT

On-Policy Self-Distillation (OPSD) has emerged as a popular paradigm for large language model (LLM) self-improvement, allowing models to act as their own teachers by leveraging privileged information such as ground-truth solutions. However, recent findings indicate that OPSD can severely degrade the performance of LLMs on complex reasoning tasks: By forcing the student to imitate an artificially confident reasoning trace conditioned on privileged information, OPSD inadvertently suppresses expressions of uncertainty and penalizes the exploratory, self-corrective behaviors required to solve challenging problems. To address this, we introduce Negative Self-Distillation (NSD), a new framework that optimizes LLMs by diverging from flawed reasoning rather than imitating privileged solutions. Instead of relying on ground-truth answers or external supervision, NSD uses the model itself to generate a question-specific negative condition (e.g., acting as a “careless reasoner”) and pushes the student’s distribution away from this self-generated negative teacher. Naively applying unlearning objectives to achieve this divergence is problematic, as flawed reasoning tokens are confounded with basic linguistic tokens; indiscriminately penalizing both risks catastrophically degrading the model’s foundational language capabilities. We resolve this by designing a dynamic gating mechanism that automatically identifies and isolates reasoning-critical tokens, ensuring gradient updates target only behavioral flaws while preserving the model’s linguistic priors. Empirically, NSD consistently outperforms OPSD and other label-free, self-bootstrapping reinforcement learning (RL) baselines. Across seven mathematical reasoning benchmarks (AIME 24/25/26, HMMT, AMC, OlympiadBench, and MATH), NSD achieves average gains of 2.3%, 7.5%, and 6.0% for 1.7B, 4B, and 8B models, respectively. Further analyses show that NSD achieves higher training efficiency while preserving the self-correction behaviors crucial for complex reasoning.

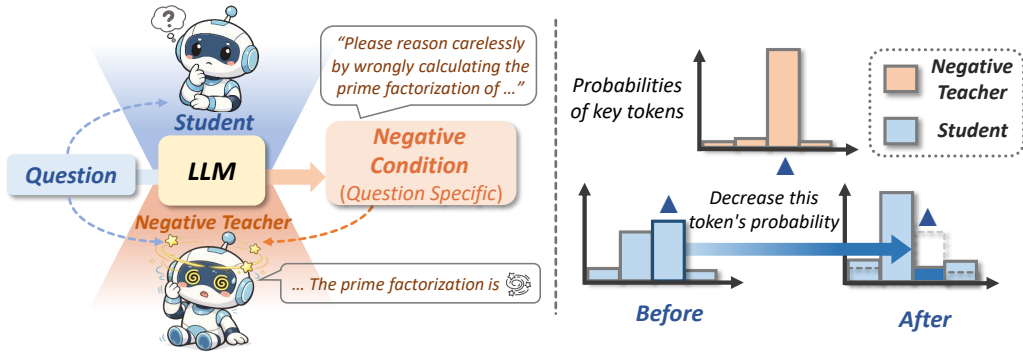


Figure 1: Overview of the NSD framework. (Left) We construct a negative teacher from the same base model via self-generated negative conditioning. (Right) The student model is optimized to diverge its distribution from that of the negative teacher.

## 1 INTRODUCTION

Reinforcement Learning with Verifiable Rewards (RLVR) (Shao et al., 2024; Yu et al., 2025; Lambert et al., 2025) has emerged as an effective paradigm for enhancing the reasoning capabilities of large language models (LLMs). However, RLVR is often bottlenecked by computational inefficiency and training signal sparsity. These challenges arise because (1) sampling multiple rollouts per query is expensive, and rollouts within a group frequently receive identical rewards on exceptionally easy or difficult problems, leading to advantage collapse and vanishing gradients (Liao et al., 2026; Xu et al., 2026a; Zhang et al., 2025b); and (2) outcome-based rewards are applied uniformly across the entire generated sequence, which obscures fine-grained, token-level credit assignment. To mitigate these limitations, On-Policy Distillation (OPD) (Agarwal et al., 2024; Lu & Lab, 2025; Song & Zheng, 2026) utilizes a stronger, external teacher model to provide dense token-level supervision over the student model’s self-sampled reasoning trajectories. While this approach successfully yields richer feedback, it introduces a practical constraint: obtaining a strictly superior external teacher that is both sufficiently capable of providing accurate dense supervision and compatible with the student’s tokenizer is often impractical.

To circumvent the reliance on external teacher models, On-Policy Self-Distillation (OPSD) (Zhao et al., 2026a; Shenfeld et al., 2026; Hübottter et al., 2026) has been proposed as a scalable alternative. In OPSD, the model acts as its own teacher by utilizing privileged information (e.g., ground-truth answers) to generate dense supervision signals for the student’s self-sampled trajectories. However, because the OPSD teacher inherently knows the ground-truth solution, it tends to produce artificially confident and highly linear reasoning trajectories (Kim et al., 2026b; Harne et al., 2026). Consequently, forcing the student to minimize the divergence from this teacher distribution inadvertently suppresses high-entropy exploration, expressions of uncertainty, and self-corrective behaviors, which are essential for complex problem-solving.

Motivated by the observation that imitating a synthetically confident oracle can degrade natural reasoning processes, we explore an alternative training paradigm: optimizing the model to explicitly avoid flawed reasoning patterns. We introduce **Negative Self-Distillation (NSD)**, a fully self-bootstrapped framework that operates without external privileged data. Instead of utilizing a teacher conditioned on the correct answer, the model is prompted to generate a question-specific negative condition (e.g., acting as a “careless reasoner”) to instantiate a negative teacher. The student is then optimized to move its token distribution away from the negative teacher, encouraging it to avoid premature conclusions and other flawed reasoning patterns. Importantly, the negative signal is generated from the model itself and does not require ground-truth solutions or external annotations.

A central challenge, however, is that not every token assigned high likelihood by the negatively conditioned teacher corresponds to a reasoning error. A naive divergence or unlikelihood objective (Welleck et al., 2020) can also penalize ordinary linguistic tokens, degrading the model’s pretrained linguistic priors. NSD therefore introduces a dynamic token-level gating mechanism that compares the negative teacher with a benign reference model and activates the negative objective only when the negative condition increases the likelihood of the sampled token. We further stabilize these updates with a bounded unlikelihood formulation and a KL-based regularization term, preventing excessive updates on high-confidence structural tokens while retaining targeted supervision on reasoning-critical tokens. This design yields a training signal that is both selective and computationally efficient: NSD requires only a single student rollout per sample, avoids full-vocabulary logit alignment, and can parallelize the reference and negative-teacher computations. Beyond accuracy, our analysis shows that NSD preserves and strengthens reflective self-correction behavior rather than encouraging overly confident, linear reasoning. Our main contributions are summarized as follows:

- We propose Negative Self-Distillation (NSD), a label-free, fully self-bootstrapped framework that enhances reasoning capabilities by optimizing the model to diverge from self-generated flawed trajectories, eliminating the need for ground-truth solutions or an external teacher.
- We introduce a token-level gating mechanism together with a bounded unlikelihood objective, enabling targeted divergence from flawed reasoning while preserving foundational language priors.
- We demonstrate that NSD consistently outperforms existing training paradigms (i.e., OPSD (Zhao et al., 2026a), Intuitor (Zhao et al., 2026b) and TTRL (Zuo et al., 2025)) across 1.7B, 4B, and 8B model sizes on seven reasoning tasks. Furthermore, NSD achieves superior training efficiency, mitigates overconfidence, and preserves the model’s intrinsic reflection capabilities.

## 2 NSD: NEGATIVE SELF-DISTILLATION

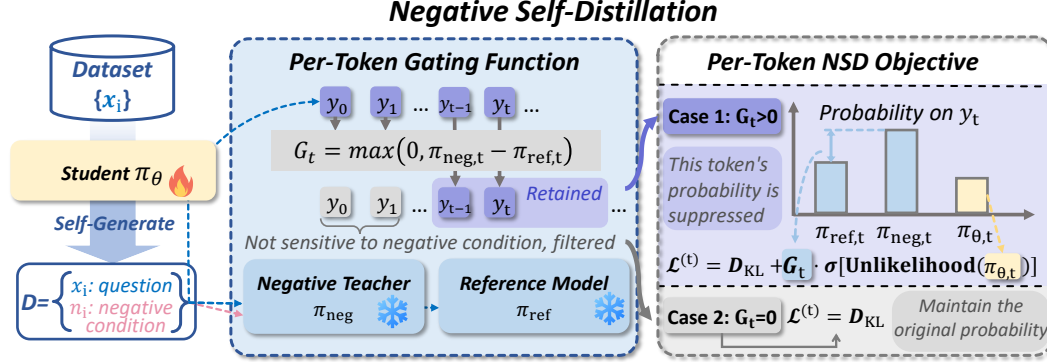


Figure 2: Overview of Negative Self-Distillation. The student model generates negative conditions from the unlabeled training data (Left). We then compare the token distributions between benign and negative contexts, isolating the tokens whose probabilities are abnormally boosted by the negative condition (Mid). Finally, the model is penalized to suppress the probabilities of these isolated tokens, while the filtered benign tokens are regularized only by KL divergence (Right).

We consider a label-free training dataset denoted as  $\mathcal{D}_{\text{raw}} = \{(x_i)\}_{i=1}^N$ , where  $x_i$  represents the problem statement. Our method consists of two core components: self negative conditioning and NSD training. We first prompt the student model  $\pi_\theta$  to generate a negative condition prompt  $n_i$  for each problem. By conditioning the model on this prompt  $n_i$ , we construct a negative teacher  $\pi_{\text{neg}}$ . We then penalize the student’s alignment with the teacher under a simple gating mechanism to avoid applying penalty to reasoning-irrelevant tokens. The overview of NSD is shown in Figure 2.

### 2.1 NEGATIVE CONDITION PROMPT GENERATION

The objective of this module is to allocate a negative instruction  $n_i$  to each training sample designed to induce flawed reasoning patterns, thereby augmenting the original  $\mathcal{D}_{\text{raw}}$  into a full negative-conditioned dataset  $\mathcal{D} = \{(x_i, n_i)\}_{i=1}^N$ . The negative condition generation strategy should follow the *self-generation* or *easy-to-get* principle, without utilizing any gold answer. By default, we adopt an online generation strategy: For a given training problem  $x$ , we first sample an initial solution  $y_{\text{init}}$  from the student model  $\pi_\theta$ . Conditioned on both the problem and this initial response, we then prompt the student model to generate an adaptive negative condition  $n$  based on its existing reasoning trace (the complete prompt is provided in Appendix C.3):

$$y_{\text{init}} \sim \pi_\theta(\cdot | x), \quad n \sim \pi_\theta(\cdot | x, y_{\text{init}}) \quad (1)$$

Our framework can naturally accommodate alternative negative condition generation strategies (discussed in Section 4.3). We default to generating negative conditions on the fly during training as it provides the most stable and effective supervision signal.

### 2.2 BACKGROUND AND CHALLENGES IN UNLIKELIHOOD TRAINING

Our motivation of the NSD training objective is to move the student’s logits distribution away from the negative teacher model’s flawed reasoning behaviors through dense token-level supervision. A natural approach to achieve this is standard unlikelihood training (Welleck et al. (2020)), which minimizes the following objective to suppress the probability of undesirable tokens:

$$\mathcal{L}_{\text{unlikelihood}} = -\log(1 - \pi_\theta(y_t | x_i, y_{<t}))$$

However, directly optimizing the objective to distance the student model from the negative teacher’s distribution presents two critical challenges and research questions (RQs):

(1) Indiscriminately treating every highly probable token under the negatively conditioned teacher as a flaw and applying the unlikelihood training is problematic, as ordinary grammatical tokens can

appear in both normal and flawed reasoning; unlearning them could easily lead to the degradation of fundamental reasoning capabilities. **RQ1:** How to identify the tokens that represent genuine reasoning flaws?

(2) This unbounded unlikelihood objective is catastrophic for highly confident, trivial tokens (e.g., punctuation or spaces) — it triggers loss explosions and overly strong gradient that destabilize training and destroy the model’s inherent logic. Specifically, as  $\pi_\theta \rightarrow 1$ , the  $\mathcal{L}_{\text{unlikelihood}}$  approaches  $\infty$  and the gradient approaches the maximum (as detailed in Appendix A). As a considerable number of tokens have a relatively high probability, this unbounded penalty triggers gradient explosions, also forcing the student to unlearn fixed fundamental linguistic priors (e.g., how to use punctuations) and rapidly update the model parameters in an unstable direction. **RQ2:** How to formulate a penalty to avoid gradient and loss explosions for training stability?

### 2.3 THE NSD TRAINING OBJECTIVE

**Token-level adaptive gating.** To address RQ1, we propose the gating mechanism to filter out grammatical tokens. During the training phase, the student model generates reasoning trajectories  $y = (y_1, \dots, y_T) \sim \pi_\theta$ . To construct the gating signals, we instantiate two frozen teacher models based on the same initial student model:

- **Reference model ( $\pi_{\text{ref}}$ ):** Conditioned only on the original problem  $x_i$ , predicting the nominal probability  $\pi_{\text{ref}}(y_t \mid x_i, y_{<t})$ .
- **Negative teacher ( $\pi_{\text{neg}}$ ):** Conditioned on both the problem and the generated negative prompt  $n_i$ , predicting the negatively biased probability  $\pi_{\text{neg}}(y_t \mid x_i, n_i, y_{<t})$ . Note that  $\pi_{\text{neg}}$  shares the same model weights as  $\pi_{\text{ref}}$ , differing only by the negative context.

We introduce a simple gating function that compares the probabilities of both models to filter out ordinary linguistic tokens and identify the tokens sensitive to the negative injection. For a given student-generated token  $y_t \sim \pi_\theta$ , the gate is defined as the adjusted positive divergence between the probability of negative and reference model on this token:

$$G_t = \max \left( 0, \pi_{\text{neg}}(y_t \mid x_i, n_i, y_{<t}) - \pi_{\text{ref}}(y_t \mid x_i, y_{<t}) \right) \quad (2)$$

The gate  $G_t \in [0, 1]$  acts as an automatic noise filter. If  $\pi_{\text{ref}} \geq \pi_{\text{neg}}$ , the token is not activated by a negative condition and naturally exempt from penalization, preserving the model’s original generative distribution. Conversely, if  $\pi_{\text{neg}} > \pi_{\text{ref}}$ , it indicates that the negative prompt has boosted the token’s likelihood, marking it as a critical target for suppression. Crucially, the penalty weight scales proportionally to this positive gap: a larger divergence directly translates to a heavier penalization.

**Gated unlikelihood penalty.** To formulate a mathematically sound penalty (i.e., the second challenge) and address RQ2, after filtering structural noise via the dynamic gate  $G_t$ , we introduce a Sigmoid-bounded unlikelihood penalty: We squash the penalty using a Sigmoid function, yielding  $\frac{1}{2 - \pi_\theta(y_t \mid x_i, y_{<t})}$ . The Gated Unlikelihood (GU) penalty is formulated as:

$$\begin{aligned} \mathcal{L}_{\text{GU}}^{(t)} &= G_t \cdot \sigma \left( -\log \left( 1 - \pi_\theta(y_t \mid x_i, y_{<t}) \right) \right) \\ &= G_t \cdot \frac{1}{2 - \pi_\theta(y_t \mid x_i, y_{<t})} \end{aligned} \quad (3)$$

This bounded formulation actively repels the student from negative flaws while safely preserving essential structural tokens. As shown in Figure 3, our sigmoid formulation allocates the strongest unlearning signals to low-to-mid confidence tokens, thereby avoiding gradient explosion on high-probability tokens. We further discuss the GU objective in detail in Section 5.3.

**Regularization and overall objective.** Let  $\pi_\theta(y_t \mid x_i, y_{<t})$  denote the current student model being optimized. Our goal is to push the student’s distribution away from the identified vulnerabilities without destroying its fundamental linguistic priors. To further regularize the objective, we introduce a point-wise forward KL penalty evaluated on the sampled token  $y_t$ . Instead of computing the full-vocabulary KL divergence, which is computationally heavy during rollouts, we apply an empirical

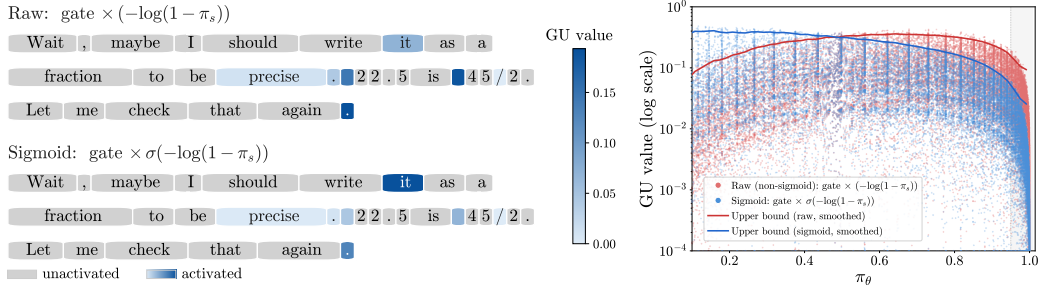


Figure 3: Left: After applying the sigmoid function, the gated unlkehood (GU) values are reduced for basic tokens (e.g., punctuations), preventing gradient explosion. Right: The  $\mathcal{L}_{\text{GU}}$  value distribution over 4,096 tokens from 100 training samples, showing that the sigmoid objective avoids penalization spikes on high-probability tokens, redistributing the  $\mathcal{L}_{\text{GU}}$  weights toward tokens with low-to-mid probabilities in the student model.

reference-weighted anchor:

$$\mathcal{L}_{\text{KL}}^{(t)} = \pi_{\text{ref}}(y_t \mid x_i, y_{<t}) \cdot \log \frac{\pi_{\text{ref}}(y_t \mid x_i, y_{<t})}{\pi_\theta(y_t \mid x_i, y_{<t})} \quad (4)$$

This is a single-sample importance-weighted estimator of  $D_{\text{KL}}(\pi_{\text{ref}} \parallel \pi_\theta)$  evaluated on the sampled token  $y_t$ . We finally formulate the NSD loss for a single token  $y_t$  as a composite objective:

$$\mathcal{L}_{\text{NSD}}^{(t)} = \mathcal{L}_{\text{GU}}^{(t)} + \alpha \cdot \mathcal{L}_{\text{KL}}^{(t)} \quad (5)$$

where  $\alpha$  is a hyperparameter. The full NSD algorithm is shown in Algorithm 1.

For every component in the  $\mathcal{L}_{\text{NSD}}$ , we validate its necessity and effectiveness through ablation studies in Section 5. The overall objective is calculated by aggregating the token-level losses across the dataset:

$$\mathcal{J}(\theta) = \mathbb{E}_{(x,n) \sim \mathcal{D}, y \sim \pi_\theta} \left[ \sum_{t=1}^{|y|} \mathcal{L}_{\text{NSD}}^{(t)} \right] \quad (6)$$

---

#### Algorithm 1 Negative Self-Distillation (NSD) Training

---

**Require:** Unlabeled dataset  $\mathcal{D}_{\text{raw}} = \{x_i\}_{i=1}^N$ , Initial model  $\pi_{\theta_0}$ , KL weight  $\alpha$

**Ensure:** Optimized student model  $\pi_\theta$

- 1: Initialize student  $\pi_\theta$ , and frozen teachers  $\pi_{\text{ref}}, \pi_{\text{neg}} \leftarrow \pi_{\theta_0}$
  - 2: **for**  $x_i \in \mathcal{D}_{\text{raw}}$  **do**
  - 3:   Sample reasoning trajectory  $y = (y_1, \dots, y_T) \sim \pi_\theta(\cdot \mid x_i)$  and the negative prompt  $n_i \sim \pi_\theta(\cdot \mid x_i, y)$
  - 4:   Initialize loss  $\mathcal{J}_i \leftarrow 0$
  - 5:   **for**  $t = 1, \dots, T$  **do**
  - 6:      $p_{\text{ref}} \leftarrow \pi_{\text{ref}}(y_t \mid x_i, y_{<t})$
  - 7:      $p_{\text{neg}} \leftarrow \pi_{\text{neg}}(y_t \mid x_i, n_i, y_{<t})$
  - 8:      $p_\theta \leftarrow \pi_\theta(y_t \mid x_i, y_{<t})$
  - 9:      $G_t \leftarrow \max(0, p_{\text{neg}} - p_{\text{ref}})$
  - 10:     $\mathcal{L}_{\text{NSD}}^{(t)} \leftarrow \frac{G_t}{2 - p_\theta} + \alpha p_{\text{ref}} \log \frac{p_{\text{ref}}}{p_\theta}$
  - 11:     $\mathcal{J}_i \leftarrow \mathcal{J}_i + \mathcal{L}_{\text{NSD}}^{(t)}$
  - 12:   **end for**
  - 13:   Update  $\theta$  using gradient  $\nabla_\theta \mathcal{J}_i$
  - 14: **end for**
  - 15: **return**  $\pi_\theta$
-



### 3 EXPERIMENTAL SETUP

**Training setup.** We use the MATH (Hendrycks et al., 2021) dataset as training dataset (for NSD, Intuitor and TTRL training, we discard the gold labels). We conduct training on the following models: Qwen3-1.7B, Qwen3-4B, and Qwen3-8B (Team, 2025). All models are trained for a total of 2 epochs, which is enough to plateau in all baselines. We set  $\alpha = 0.01$ , top- $k = 32$ , batch size = 32. For NSD, we set the max generation length to 4096.

**Evaluation.** We evaluate the math reasoning ability of all models on the following benchmarks: AIME 2024, AIME 2025, AIME 2026, HMMT 2025 (Dekoninck et al., 2026), MATH-500, AMC 2023 and OlympiadBench (He et al., 2024). For OlympiadBench, we exclude the proof problems. By default, we set hyperparameters according to the recommended setting in Qwen3 report (Team, 2025): temperature = 0.6; top- $p = 0.95$ ; top- $k = 20$ . The output length is set to 32K.

**Baselines.** We compare with the following methods representing three different training paradigms: **OPSD** (Zhao et al., 2026a): A standard distillation framework that minimizes the full-vocabulary KL divergence between the student and a teacher conditioned on the gold solution. **Intuitor** (Zhao et al., 2026b): A representative RLIF (Reinforcement Learning from Internal Feedback) implementation, which is a variant of GRPO and utilizes average confidence (self-certainty) as the intrinsic reward. **TTRL** (Zuo et al., 2025): A variant of GRPO that utilizes the majority-voting consensus as pseudo-gold labels. While vanilla TTRL typically optimizes directly on the test set, we apply it to the training dataset to ensure a fair comparison with other baseline methods.

A conceptual comparison of NSD with existing related methods, along with their implementation details and prompt templates, is provided in Appendix B and C.

### 4 EVALUATION RESULTS

In this section, we first present the main experimental results across multiple mathematical reasoning benchmarks (§4.1). Then we empirically demonstrate NSD achieves better training efficiency and promotes reflection abilities compared to other baselines (§4.2, §4.4). Finally, we show that employing simpler negative conditioning strategies in NSD can also yield comparable effectiveness (§4.3).

#### 4.1 MAIN RESULTS

The main results are shown in Table 1. We highlight the following key observations:

**NSD achieves the overall best performance on the models with different sizes.** As shown in Table 1, NSD consistently achieves the highest average improvements across all model scales, yielding  $\Delta$  Avg gains of **+2.3%**, **+7.5%**, and **+6.0%** on the three models respectively. While baselines like OPSD<sup>†</sup> and RL excel narrowly on AIME 2024, our 4B and 8B models achieve broader generalization across diverse math tasks, maintaining peak AIME accuracies of 35.8% and 39.6%. Notably, unlike other baselines where small improvements possibly partly stem from randomness, NSD guarantees stable performance gains, supported by a significantly low  $p$ -value.

**NSD is more promising on larger model sizes due to self-generated negative conditions.** An observation from Table 1 is that NSD exhibits stronger performance gains on larger models compared to the smaller 1.7B variant. This scaling behavior is tied to our online negative condition generation mechanism: NSD uses on the model itself to generate solution-specific negative conditions. By optimizing against these higher-quality conditions, larger models receive a stronger contrastive training signal, which translates into substantial improvements on challenging reasoning tasks.

**Why does NSD outperform other baselines?** Compared to OPSD, label-free training of NSD without the privileged information prevents bias (e.g., reinforcing reasoning shortcuts due to the gold solution) and reflection collapse caused by overconfidence (Kim et al., 2026b). We provide additional analysis in Section 4.2 that further confirms NSD better promotes reflection behaviors than other methods. Case studies in Appendix D.4 also illustrate how NSD-trained models abandon the wrong reasoning trajectory and switch to the right one. Compared to Intuitor and TTRL which use model

Table 1: Main evaluation results on mathematical reasoning benchmarks. We report the Avg@8 (%) performance under non-thinking mode (we report the performance under thinking mode in Appendix D.2).  $\Delta$  Avg is the average absolute improvement over the same-size baseline across all 7 benchmarks. We report the best checkpoint on the validation set within 2 training epochs. **Bold** marks the best result in each model-size group; underline marks the second best.  $\dagger$  denotes methods that require ground-truth labels. The last two columns report the 95% CI and one-sided  $p$ -value (which measures the probability of observing an improvement at least as large as the really observed one if the improvement were due to chance) for  $\Delta$  Avg@8, respectively. OlympiadBench is evaluated on the 675 open-ended math problems (excluding proof problems) using the official judge with symbolic comparison.

| Method             | AIME 2024   | AIME 2025   | AIME 2026   | HMMT 2025 Feb | AMC 2023    | Olympiad-Bench | MATH-500    | $\Delta$ Avg | 95% CI       | $p$         |
|--------------------|-------------|-------------|-------------|---------------|-------------|----------------|-------------|--------------|--------------|-------------|
| <i>1.7B Models</i> |             |             |             |               |             |                |             |              |              |             |
| Qwen3-1.7B         | 9.6         | 10.0        | 9.6         | 7.1           | 44.1        | 37.1           | 62.5        | —            | —            | —           |
| OPSD $\dagger$     | <b>15.0</b> | <u>14.2</u> | 8.8         | 5.8           | 44.1        | <u>37.2</u>    | 62.5        | +1.1         | [−0.3, +2.4] | 0.06        |
| Intuitior          | 13.8        | 8.3         | 8.3         | 6.7           | 43.4        | 35.4           | 60.6        | −0.5         | [−1.8, +0.8] | 0.29        |
| TTRL               | 11.3        | 11.3        | <u>9.6</u>  | <b>8.3</b>    | 41.6        | 36.8           | <b>63.4</b> | +0.3         | [−1.0, +1.5] | 0.29        |
| <b>NSD</b>         | <u>14.2</u> | <b>17.9</b> | <b>10.0</b> | <u>7.1</u>    | <b>45.9</b> | <b>38.7</b>    | <u>62.6</u> | <b>+2.3</b>  | [+0.7, +4.0] | 0.001       |
| <i>4B Models</i>   |             |             |             |               |             |                |             |              |              |             |
| Qwen3-4B           | 23.8        | 20.4        | 17.9        | 10.8          | 68.8        | 47.8           | 71.2        | —            | —            | —           |
| OPSD $\dagger$     | 25.4        | 22.5        | 15.8        | <u>15.8</u>   | 68.8        | 47.6           | <u>71.7</u> | +1.0         | [−0.3, +2.4] | 0.10        |
| Intuitior          | 24.6        | <u>25.8</u> | <u>18.3</u> | 13.8          | <u>70.0</u> | <u>47.7</u>    | 69.8        | +1.3         | [−0.5, +3.1] | 0.05        |
| TTRL               | <u>25.8</u> | 19.6        | <u>18.3</u> | 11.7          | 68.1        | 47.1           | 71.7        | +0.2         | [−1.2, +1.7] | 0.33        |
| <b>NSD</b>         | <b>35.8</b> | <b>31.3</b> | <b>29.2</b> | <b>16.3</b>   | <b>76.3</b> | <b>51.0</b>    | <b>73.1</b> | <b>+7.5</b>  | [+5.4, +9.5] | $< 10^{-4}$ |
| <i>8B Models</i>   |             |             |             |               |             |                |             |              |              |             |
| Qwen3-8B           | 28.8        | 19.2        | 18.3        | 11.7          | 67.2        | 48.9           | 73.1        | —            | —            | —           |
| OPSD $\dagger$     | 30.0        | <u>21.3</u> | 17.1        | 12.1          | 66.9        | 48.2           | <u>73.5</u> | +0.3         | [−1.3, +1.9] | 0.33        |
| Intuitior          | 34.6        | 20.4        | <u>18.3</u> | <u>13.8</u>   | <u>70.9</u> | <u>49.5</u>    | 72.7        | +1.9         | [+0.2, +3.4] | 0.02        |
| TTRL               | 29.2        | 18.3        | 17.1        | 10.8          | 69.1        | 49.1           | 73.0        | −0.1         | [−1.4, +1.3] | 0.57        |
| <b>NSD</b>         | <b>39.6</b> | <b>26.3</b> | <b>25.0</b> | <b>17.9</b>   | <b>75.6</b> | <b>50.6</b>    | <b>74.1</b> | <b>+6.0</b>  | [+4.0, +7.9] | $< 10^{-4}$ |

confidence or majority voting to generate training signals, NSD removes the reliance on the model’s self-judgement ability, which leads to possible incorrect training signals. For example, weaker models hardly gain improvement from Intuitior (−0.5% on Qwen3-1.7B) because their high confidence does not necessarily equate to high accuracy. Furthermore, those confidence-based bootstrapping methods also degrade the reflection ability, as shown in Section 4.2.

## 4.2 NSD INSPIRES REFLECTION

**NSD prevents over-confidence and preserves exploratory reflection.** We evaluate model reflection capabilities by measuring the average frequency of reflection tokens (e.g., “Wait”) across AIME and HMMT benchmarks (Table 2). The detailed definition of reflection tokens is shown in Appendix E. We observe that OPSD and Intuitior severely suppress reflective behavior (dropping to 2.18 and 0.75 per response, respectively), as training on ground-truth or unverified positive rollouts encourages overly direct, non-verifying reasoning trajectories.

Table 2: The average reflection token frequency per response on Qwen3-4B.

| Method     | AIME 2024  | AIME 2025  | HMMT 2025  | Average    |
|------------|------------|------------|------------|------------|
| Baseline   | 6.8        | 2.2        | 1.7        | 3.6        |
| OPSD       | 2.6        | 2.1        | 1.8        | 2.2        |
| Intuitior  | 0.6        | 1.0        | 0.7        | 0.8        |
| <b>NSD</b> | <b>6.9</b> | <b>7.5</b> | <b>8.1</b> | <b>7.5</b> |

Conversely, NSD substantially enhances reflection frequency (yielding up to 7.5 per response). By penalizing flawed reasoning paths, NSD avoids over-confidence and enables the model to autonomously re-evaluate potential errors during complex inference, which is also demonstrated by our case study in Appendix D.4.

### 4.3 NEGATIVE CONDITION VARIANTS STUDY

In our main experiments, we default to an online self negative condition generation strategy (denoted as online strategy briefly). While intuitively well-motivated, this approach incurs computational overhead from online rollouts. To explore more efficient alternatives, we investigate the impact of simpler conditioning strategies, selecting the variants based on effective LLM negative conditioning paradigms identified in (Chatziveroglou et al., 2025). In this section, we discuss the following offline generation strategies while our primary evaluations in the previous sections are conducted using the online paradigm:

Strategy 1: Solution-aware negative conditioning. Besides inputting the question, we let the student model rollout first, then prompt it to generate a negative condition based on the question and rollout.

Strategy 2: Question-only negative conditioning. Input the training sample question to the frozen initial student and prompt it to generate a possible negative condition based on it.

Strategy 3: Noise conditioning. Simply add irrelevant Wikipedia articles as noise (denoted as wiki-irr strategy; “irr” stands for irrelevant).

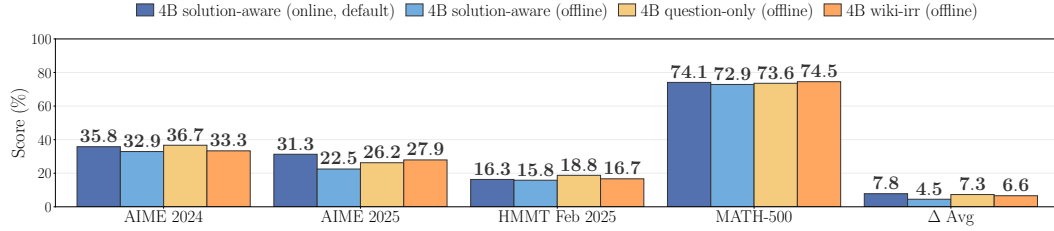


Figure 4: Evaluation results of NSD across different conditioning strategies.  $\Delta$  denotes the average absolute improvement over the base model across these 4 datasets. The dashed line represents the reference  $\Delta$  achieved by the default online strategy.

We evaluate the three NSD conditioning strategies on the Qwen3-4B model. The experimental result of different conditioning strategies is shown in Figure 4. Notably, the question-only negative conditioning strategy achieves a 7.3% average improvement, comparable to 7.8% using our default online solution-aware approach. Furthermore, even the most lightweight offline strategy (wiki-irr) also performs competitively with our default approach, highlighting NSD’s broad scalability to diverse and efficient negative conditions. In contrast, the offline solution-aware strategy exhibits relatively lower performance, primarily driven by its reliance on outdated offline-generated solutions during conditioning.

### 4.4 EFFICIENCY OF NSD

**NSD exhibits superior training efficiency compared to OPSD and RLIE.** We focus our detailed latency analysis on the computational overhead of the rollout phase, which is the dominant source of training time discrepancy across different algorithms.

In the rollout stage, the student model samples batch size  $\times n$  rollouts, where  $n$  denotes the number of samples per prompt. While GRPO-based baselines (Intuitior and TTRL) demand  $n = 8$ , both NSD and OPSD require only  $n = 1$ . Subsequently, OPSD and NSD perform additional forward passes on the generated sequences: OPSD prefills each concatenated prompt-response pair to extract top- $k$  log-probabilities, where  $k = 32$  in NSD and 128 in OPSD; online NSD generates an online negative condition based on the student’s solution before running two forward passes to compute  $\pi_{\text{ref}}$  and  $\pi_{\text{neg}}$ .

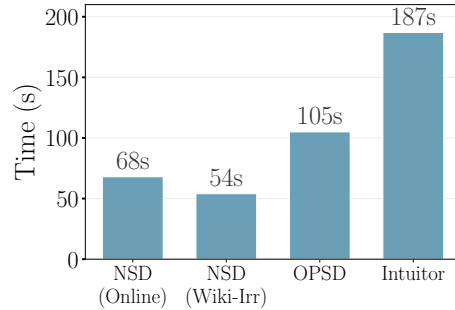


Figure 5: Average wall-clock time per training step with 6 or 8 A100 GPUs. For NSD and OPSD, the student model occupies 4 GPUs and the teacher occupies 2 GPUs. For Intuitior, the generation stage is executed across all 8 GPUs. Notably, the wiki-irr strategy effectively reduces the latency compared to using the default online rollout in NSD.



The time consumption is shown in Figure 5. Overall, NSD achieves superior training efficiency through three primary factors: (1) **Minimal Rollout Overhead:** Unlike multi-sample GRPO-style baselines, NSD requires only a single rollout per sample, reducing rollout time by  $\sim 60\%$ . Furthermore, static negative condition generation strategies (e.g., wiki-irr) save online rollout time entirely, lowering the overall latency from 68s to 54s. (2) **Parallelized Forward Prefilling:** Although computing  $\pi_{\text{ref}}$  and  $\pi_{\text{neg}}$  involves two distinct prompts, both share the same model weights and can be prefilled concurrently in parallel. (3) **Scalar-Only Loss Computation:** NSD requires only three scalar token probabilities, avoiding full-vocabulary logit projections. The result shows that NSD trains faster overall than OPSD, proving that our parallelized prefilling costs substantially less than OPSD’s Top- $k$  logit alignment.

#### 4.5 MORE EVALUATIONS AND ANALYSES

We conduct several supplementary evaluations provided in Appendix D. First, we evaluate our models under the Pass@8 metric in Appendix D.1. Second, we report the performance under thinking mode in Appendix D.2. NSD continues to outperform all baselines under these settings. Third, in Appendix D.3, we investigate an alternative objective formulation that treats the negative of the loss as an advantage signal for policy-gradient optimization, demonstrating that the NSD framework is scalable to policy-gradient-style training paradigms.

### 5 UNDERSTANDING NSD TRAINING OBJECTIVE

#### 5.1 ADAPTIVE GATING ANALYSIS

The adaptive gating function is designed to filter out trivial tokens while retaining essential ones. RLCS (Pan et al., 2026) rigorously conceptualizes this by categorizing tokens into style and task tokens, treating the former as noise. A detailed definition is shown in Appendix E. To evaluate how effectively the NSD gating function and existing weighting methods filter out style tokens, we sample a subset of 100 training queries and analyze the logit distributions across the initial 4,096 tokens and calculate the style-task ratio ( $\mathcal{T}$  denotes the set of task tokens and  $\mathcal{S}$  denotes the set of style tokens):

$$R = \left[ \frac{1}{|\mathcal{S}|} \sum_{t \in \mathcal{S}} w_t \right] / \left[ \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} w_t \right] \quad (7)$$

We compare NSD adaptive gating with initial ratio, entropy-based OPSD weighting (Wang et al., 2026b), and vanilla OPSD loss (Zhao et al., 2026a):

NSD:  $w_t = \max(0, \pi_{\text{neg}}(y_t | x, a, y_{<t}) - \pi_{\text{ref}}(y_t | x, y_{<t}))$ ; Entropy-OPSD:  $w_t = -\sum_v \pi_\theta(v | x, y_{<t}) \log \pi_\theta(v | x, y_{<t})$ ; OPSD:  $w_t = \sum_v \pi_\theta(v) \log \frac{\pi_\theta(v)}{\pi_{\text{gold}}(v)}$ .

A lower  $R$  inherently signifies a better approach (Pan et al., 2026); it implies the method prioritizes task tokens, suppressing gradient generation on meaningless tokens — previous work (Pan et al., 2026; Zhao et al., 2026a) shows that in OPSD, the training signal might be dominated by style tokens, causing the student to imitate styles rather than learning reasoning ability. As shown in Table 3, the NSD gating mechanism alone filters style tokens more effectively than both entropy-based and OPSD-loss-based weighting methods.

Table 3: Comparison of style-task ratio across different methods. A lower value indicates a better approach (Pan et al., 2026).

| Method                  | NSD gate<br>(wiki) | NSD gate<br>(solution-aware) | NSD gate<br>(question-only) | Entropy-<br>OPSD | OPSD |
|-------------------------|--------------------|------------------------------|-----------------------------|------------------|------|
| <b>style-task ratio</b> | 2.6×               | 3.4×                         | 3.5×                        | 3.9×             | 5.4× |

## 5.2 KL ABLATION

We investigate the necessity of the KL divergence constraint within the NSD framework. Figure 6 illustrates this via an ablation study comparing the standard NSD against a variant without the KL anchor (NSD-noKL).

Removing the KL constraint leads to a mid-training collapse. As shown in Figure 6 (left), NSD-noKL drastically shifts the student’s distribution, inducing a cycle of learning and forgetting, evidenced by sharp oscillations in the curve. Furthermore, Figure 6 (right) demonstrates that the gate activation ratio in NSD-noKL initially increases but drops precipitously around the 120th step, coinciding precisely with the KL collapse. We also observe that the mean  $\mathcal{L}_{GU}$  value decreases significantly, indicating that the gating mechanism activates spuriously and loses its effectiveness—a direct result of the model drifting excessively from the reference without KL regularization.

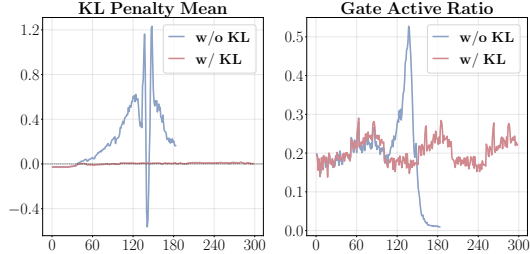


Figure 6: Training log of NSD w/ and w/o KL constraint on Qwen3-4B. Left: Forward KL between the reference model and student model per training step; Right: The average activated gating ( $G_t$ ) per training step.

## 5.3 DISCUSSION ON GATED UNLIKELYHOOD

A natural inherent consequence of the gating formulation is its sensitivity to minor probability fluctuations in highly predictable tokens (where  $\pi_{\text{neg}} \approx \pi_{\text{ref}} \rightarrow 1$ , usually trivial tokens like punctuations). Occasionally, inherent variance may cause the negative teacher model to assign a marginally higher probability than the reference model, bypassing the filter (e.g., probabilities for space tokens often fluctuate slightly around 99%). Nevertheless, this artifact is controlled: the minuscule divergence yields a near-zero gate value  $G_t$ , ensuring that the overall gating on these high-confidence tokens remains negligible.

To further avoid distancing from these tokens, we bound the pure unlikelihood penalty via Sigmoid function (Equation 3), so that the model enjoys implicit gradient attenuation. The Sigmoid penalty serves as a structural failsafe against gating imperfections. As shown in Figure 3,  $\mathcal{L}_{GU}$  value on high-probability tokens is significantly lower than the value of pure unlikelihood objectives. Gradient analysis is mathematically discussed in Appendix A.

## 6 RELATED WORK

**On policy distillation.** The original OPD (Agarwal et al., 2024; Lu & Lab, 2025; Song & Zheng, 2026) relies on external reward models. Self distillation (Zhao et al., 2026a; Hübottter et al., 2026; Shenfeld et al., 2026) removes external teachers by using ground-truth solutions as hints, but suffers from solution bias and overconfidence (Kim et al., 2026b; Harne et al., 2026; Wang et al., 2026a); Recent studies have increasingly optimized the distillation method across various dimensions, mainly including weak supervision (He et al., 2026; Li et al., 2026), credit assignment (Pan et al., 2026; Wang et al., 2026d; Xu et al., 2026c; Wang et al., 2026c), agentic scenarios (Wu et al., 2026; Lu et al., 2026), and other better learning objectives (Yang et al., 2026; Heo et al., 2026; Jiang et al., 2026; Shen et al., 2026; Kim et al., 2026a).

**Label-free reinforcement learning.** Existing label-free training methods primarily rely on substituting rewards with self-generated ones (usually based on confidence or entropy) within RLVR frameworks (Zhao et al., 2026b; Li et al., 2025; Yuan et al., 2025; Prabhudesai et al., 2026; Huang et al., 2026b;a) or OPD frameworks (Gkountouras et al., 2026; Li et al., 2026), as well as generating gold labels by the model itself (Zhang et al., 2025a; Zuo et al., 2025).

**Training with negative signals.** Unlikelihood objective (Welleck et al., 2020; Li et al., 2020) has been proposed to train earlier small language models. Several studies incorporate both positive and negative trajectories into distillation or RLVR frameworks (Xu et al., 2026b; Hamdan & Yuret, 2025; Yang et al., 2024). Notably, NSR (Zhu et al., 2026) explores RLVR training driven exclusively by

negative signals, demonstrating that it can preserve high-confidence priors while mitigating overfitting. Furthermore, in the context of self-distillation, recent works introduce negative signals to alleviate student overconfidence (Shen et al., 2026; Kim et al., 2026a).

## 7 CONCLUSION

In this work, we propose Negative Self-Distillation (NSD), a label-free training framework comprising negative conditioning and gated unlikelihood training. Empirical results demonstrate that NSD consistently outperforms existing baselines across seven mathematical reasoning benchmarks under various model sizes. Comprehensive analyses and ablation studies show that our adaptive gating mechanism effectively isolates genuinely flawed tokens, while the sigmoid unlikelihood objective ensures smoother reasoning gradients. Furthermore, NSD accommodates diverse negative conditioning strategies, establishing it as a highly scalable framework. Its training efficiency is enhanced by bypassing full-vocabulary computations and leveraging parallelized forward passes for the negative teacher and reference model. Importantly, NSD inherently preserves and stimulates the model’s capacity for self-reflection, highlighting its potential as a promising post-training method for enhancing the reasoning capabilities of LLMs.

## LIMITATIONS

NSD relies on the student model’s inherent capacity to generate negative conditions. Consequently, this approach may be less effective for extremely small or weak models that struggle to produce meaningful negative contrasts for optimization. However, given the rapid capability scaling of modern foundational models, this capacity bottleneck is expected to diminish naturally in future architectures or in stronger models.

Under the online strategy, NSD requires negative-condition generation and two forward passes through the two same frozen models. Nevertheless, we explored alternative conditioning strategies, including efficient generation-free methods like the wiki-irr strategy, which can mitigate the rollout costs while maintaining competitive performance. Moreover, executing the two forward passes in parallel at each training step effectively minimizes overall wall-clock latency.

## ACKNOWLEDGMENTS

This research is partially funded by the NVIDIA Academic Grant and Amazon Research Award. We thank Xinyu Wang and Yu Gu for their valuable feedback and suggestions, particularly for the experimental design.

## REFERENCES

- Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=3zKtaqxLhW>.
- Giannis Chatziveroglou, Richard Yun, and Maura Kelleher. Exploring LLM reasoning through controlled prompt variations, 2025. URL <https://arxiv.org/abs/2504.02111>.
- Jasper Dekoninck, Nikola Jovanović, Tim Gehrunger, Kári Rognvaldsson, Ivo Petrov, Chenhao Sun, and Martin Vechev. Beyond benchmarks: MathArena as an evaluation platform for mathematics with LLMs, 2026. URL <https://arxiv.org/abs/2605.00674>.
- Mukesh Ghimire, Aosong Feng, Liwen You, Youzhi Luo, Fang Liu, and Xuan Zhu. PRISM: A unified framework for post-training LLMs without verifiable rewards, 2026. URL <https://arxiv.org/abs/2601.04700>.
- John Gkountouras, Josip Jukić, and Ivan Titov. Consensus as privileged context for label-free self-distillation, 2026. URL <https://arxiv.org/abs/2607.13643>.

- Shadi Hamdan and Deniz Yuret. How much do LLMs learn from negative examples?, 2025. URL <https://arxiv.org/abs/2503.14391>.
- Sarthak Harne, Chinmay Karkar, Yash Pandya, Ahmed Awadallah, and Akshay Nambi. Privileged, but biased: How pi-conditioned teachers break self-distillation, 2026. URL <https://arxiv.org/abs/2608.04794>.
- Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Leng Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, Jie Liu, Lei Qi, Zhiyuan Liu, and Maosong Sun. Olympiad-Bench: A challenging benchmark for promoting AGI with olympiad-level bilingual multimodal scientific problems, 2024. URL <https://arxiv.org/abs/2402.14008>.
- Yinghui He, Simran Kaur, Adithya Bhaskar, Yongjin Yang, Jiarui Liu, Narutatsu Ri, Liam Fowl, Abhishek Panigrahi, Danqi Chen, and Sanjeev Arora. Self-Distillation Zero: Self-revision turns binary rewards into dense supervision, 2026. URL <https://arxiv.org/abs/2604.12002>.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset, 2021. URL <https://arxiv.org/abs/2103.03874>.
- Byeongho Heo, Jaehui Hwang, Sangdoo Yun, and Dongyoon Han. On-policy delta distillation, 2026. URL <https://arxiv.org/abs/2607.15161>.
- Chengsong Huang, Haolin Liu, Tong Zheng, Runpeng Dai, Langlin Huang, Jinyuan Li, Zongxia Li, Zhepei Wei, Yu Meng, and Jiabin Huang. G-Zero: Self-play for open-ended generation from zero data. *arXiv preprint arXiv:2605.09959*, 2026a.
- Chengsong Huang, Wenhao Yu, Xiaoyang Wang, Hongming Zhang, Zongxia Li, Ruosen Li, Jiabin Huang, Haitao Mi, and Dong Yu. R-Zero: Self-evolving reasoning LLM from zero data. In *The Fourteenth International Conference on Learning Representations*, 2026b. URL <https://openreview.net/forum?id=96apU6YzSO>.
- Jonas Hübner, Frederike Lübeck, Lejs Behric, Anton Baumann, Marco Bagatella, Daniel Marta, Ido Hakimi, Idan Shenfeld, Thomas Kleine Buening, Carlos Guestrin, and Andreas Krause. Reinforcement learning via self-distillation, 2026. URL <https://arxiv.org/abs/2601.20802>.
- Li Jiang, Haoran Xu, Yichuan Ding, and Amy Zhang. Trajectory-refined distillation, 2026. URL <https://arxiv.org/abs/2606.08432>.
- Jeonghye Kim, Jiwon Jeon, Dongsheng Li, and Yuqing Yang. Rebellious student: Reversing teacher signals for reasoning exploration with self-distilled RLVR, 2026a. URL <https://arxiv.org/abs/2605.10781>.
- Jeonghye Kim, Xufang Luo, Minbeom Kim, Sangmook Lee, Dohyung Kim, Jiwon Jeon, Dongsheng Li, and Yuqing Yang. Why does self-distillation (sometimes) degrade the reasoning capability of LLMs?, 2026b. URL <https://arxiv.org/abs/2603.24472>.
- Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, Yuling Gu, Saumya Malik, Victoria Graf, Jena D. Hwang, Jiangjiang Yang, Ronan Le Bras, Øyvind Taffjord, Chris Wilhelm, Luca Soldaini, Noah A. Smith, Yizhong Wang, Pradeep Dasigi, and Hannaneh Hajishirzi. Tulu 3: Pushing frontiers in open language model post-training, 2025. URL <https://arxiv.org/abs/2411.15124>.
- Margaret Li, Stephen Roller, Ilya Kulikov, Sean Welleck, Y-Lan Boureau, Kyunghyun Cho, and Jason Weston. Don’t say that! making inconsistent dialogue unlikely with unlikelihood training. In Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault (eds.), *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 4715–4728, Online, July 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.428. URL <https://aclanthology.org/2020.acl-main.428/>.

- Pengyi Li, Matvey Skripkin, Alexander Zubrey, Andrey Kuznetsov, and Ivan Oseledets. Confidence is all you need: Few-shot RL fine-tuning of language models, 2025. URL <https://arxiv.org/abs/2506.06395>.
- Yijiang Li, Bingyang Wang, Yijun Liang, Yunjie Tian, Di Fu, and Nuno Vasconcelos. On-policy self-distillation without any supervision, 2026. URL <https://arxiv.org/abs/2608.06296>.
- Baohao Liao, Hanze Dong, Xinxing Xu, Christof Monz, and Jiang Bian. Self-hinting language models enhance reinforcement learning, 2026. URL <https://arxiv.org/abs/2602.03143>.
- Kevin Lu and Thinking Machines Lab. On-policy distillation. *Thinking Machines Lab: Connectionism*, 2025. doi: 10.64434/tml.20251026. <https://thinkingmachines.ai/blog/on-policy-distillation>.
- Zhengxi Lu, Zhiyuan Yao, Zhuowen Han, Zi-Han Wang, Jinyang Wu, Qi Gu, Xunliang Cai, Weiming Lu, Jun Xiao, Yueting Zhuang, and Yongliang Shen. Self-distilled agentic reinforcement learning, 2026. URL <https://arxiv.org/abs/2605.15155>.
- Leyi Pan, Shuchang Tao, Yunpeng Zhai, Lingzhe Zhang, Zhaoyang Liu, Bolin Ding, Aiwei Liu, and Lijie Wen. RLCSO: Reinforcement learning with contrastive on-policy self-distillation, 2026. URL <https://arxiv.org/abs/2606.11709>.
- Mihir Prabhudesai, Lili Chen, Alex Ippoliti, Katerina Fragkiadaki, Hao Liu, and Deepak Pathak. Maximizing confidence alone improves reasoning, 2026. URL <https://openreview.net/forum?id=Qhg479eBmo>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models, 2024. URL <https://arxiv.org/abs/2402.03300>.
- Guobin Shen, Xiang Cheng, Chenxiao Zhao, Lei Huang, Jindong Li, Dongcheng Zhao, and Xing Yu. Anti-self-distillation for reasoning RL via pointwise mutual information, 2026. URL <https://arxiv.org/abs/2605.11609>.
- Idan Shenfeld, Mehul Damani, Jonas Hübner, and Pulkit Agrawal. Self-distillation enables continual learning, 2026. URL <https://arxiv.org/abs/2601.19897>.
- Mingyang Song and Mao Zheng. A survey of on-policy distillation for large language models. *arXiv preprint arXiv:2604.00626*, 2026.
- Qwen Team. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Meng Wang, Haohan Zhao, Wenzhuo Liu, Lu Yang, Geng Liu, Haiyang Guo, Guo-Sen Xie, Gaofeng Meng, Hongbin Liu, and Fei Zhu. Denser  $\neq$  better: Limits of on-policy self-distillation for continual post-training, 2026a. URL <https://arxiv.org/abs/2607.01763>.
- Shenzhi Wang, Le Yu, Chang Gao, Chujie Zheng, Shixuan Liu, Rui Lu, Kai Dang, Xiong-Hui Chen, Jianxin Yang, Zhenru Zhang, Yuqiong Liu, An Yang, Andrew Zhao, Yang Yue, Shiji Song, Bowen Yu, Gao Huang, and Junyang Lin. Beyond the 80/20 rule: High-entropy minority tokens drive effective reinforcement learning for LLM reasoning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026b. URL <https://openreview.net/forum?id=yfcpdY4gMP>.
- Yuanyi Wang, Su Lu, Yanggan Gu, Pengkai Wang, Yifan Yang, Zhaoyi Yan, Congkai Xie, Jianmin Wu, and Hongxia Yang. Not all disagreement is learnable: Token teachability in on-policy distillation, 2026c. URL <https://arxiv.org/abs/2605.26844>.
- Zi-Han Wang, Zhengxi Lu, Zhiyuan Yao, Jinyang Wu, Jie Wu, Zhengzhou Cai, Yueqing Sun, Ziang Ye, Linji Hao, Qi Gu, Xunliang Cai, Yongliang Shen, and Yujiu Yang. AgentOPSD: Recursive self-distillation for agentic reinforcement learning, 2026d. URL <https://arxiv.org/abs/2608.05987>.



- Sean Welleck, Ilya Kulikov, Stephen Roller, Emily Dinan, Kyunghyun Cho, and Jason Weston. Neural text generation with unlikelihood training. In *International Conference on Learning Representations*, 2020. URL <https://openreview.net/forum?id=SJeYe0NtvH>.
- Jinyang Wu, Shuo Yang, Zhengxi Lu, Fan Zhang, Yuhao Shen, Lang Feng, Haoran Luo, Zheng Lian, Shuai Zhang, Zhengqi Wen, and Jianhua Tao. SEED: Self-evolving on-policy distillation for agentic reinforcement learning, 2026. URL <https://arxiv.org/abs/2607.14777>.
- Haobo Xu, Sirui Chen, Ruizhong Qiu, Yuchen Yan, Chen Luo, Monica Xiao Cheng, Jingrui He, and Hanghang Tong. Prune as you generate: Online rollout pruning for faster and better RLVR. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (eds.), *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 13876–13893, San Diego, California, United States, July 2026a. Association for Computational Linguistics. ISBN 979-8-89176-390-6. doi: 10.18653/v1/2026.acl-long.632. URL <https://aclanthology.org/2026.acl-long.632/>.
- Shuyao Xu, Cheng Peng, Jiangxuan Long, Weidi Xu, Wei Chu, and Yuan Qi. Harnessing negative signals: Reinforcement distillation from teacher data for LLM reasoning. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (eds.), *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1618–1639, San Diego, California, United States, July 2026b. Association for Computational Linguistics. ISBN 979-8-89176-390-6. doi: 10.18653/v1/2026.acl-long.74. URL <https://aclanthology.org/2026.acl-long.74/>.
- Yuanda Xu, Hejian Sang, Zhengze Zhou, Ran He, Zhipeng Wang, and Alborz Geramifard. TIP: Token importance in on-policy distillation, 2026c. URL <https://arxiv.org/abs/2604.14084>.
- Kevin Yang, Dan Klein, Asli Celikyilmaz, Nanyun Peng, and Yuandong Tian. RLCD: Reinforcement learning from contrastive distillation for LM alignment. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=v3XXtxWKi6>.
- Shenzhi Yang, Guangcheng Zhu, Bowen Song, Haobo Wang, Mingxuan Xia, Xing Zheng, Yingfan Ma, Zhongqi Chen, Weiqiang Wang, Junbo Zhao, and Gang Chen. OPRD: On-policy representation distillation, 2026. URL <https://arxiv.org/abs/2606.06021>.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. DAPO: An open-source LLM reinforcement learning system at scale, 2025. URL <https://arxiv.org/abs/2503.14476>.
- Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models, 2025. URL <https://arxiv.org/abs/2401.10020>.
- Kongcheng Zhang, QI YAO, Shunyu Liu, Yingjie Wang, Baisheng Lai, Jieping Ye, Mingli Song, and Dacheng Tao. Consistent paths lead to truth: Self-rewarding reinforcement learning for LLM reasoning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025a. URL <https://openreview.net/forum?id=ckW701s93V>.
- Xingjian Zhang, Siwei Wen, Wenjun Wu, and Lei Huang. EDGE-GRPO: Entropy-driven GRPO with guided error correction for advantage diversity, 2025b. URL <https://arxiv.org/abs/2507.21848>.
- Siyan Zhao, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover. Self-Distilled Reasoner: On-policy self-distillation for large language models, 2026a. URL <https://arxiv.org/abs/2601.18734>.



Xuandong Zhao, Zhewei Kang, Aosong Feng, Sergey Levine, and Dawn Song. Learning to reason without external rewards. In *The Fourteenth International Conference on Learning Representations*, 2026b. URL <https://openreview.net/forum?id=OU9nFEYR2M>.

Xinyu Zhu, Mengzhou Xia, Zhepei Wei, Wei-Lin Chen, Danqi Chen, and Yu Meng. The surprising effectiveness of negative reinforcement in LLM reasoning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=ftVlLG9cks>.

Yuxin Zuo, Kaiyan Zhang, Li Sheng, Shang Qu, Ganqu Cui, Xuekai Zhu, Haozhan Li, Yuchen Zhang, Xinwei Long, Ermo Hua, Biqing Qi, Youbang Sun, Zhiyuan Ma, Lifan Yuan, Ning Ding, and Bowen Zhou. TTRL: Test-time reinforcement learning, 2025. URL <https://arxiv.org/abs/2504.16084>.

## A ANALYSIS OF CANDIDATE GATED UNLIKELYHOOD GRADIENTS IN NSD AND OPSD OBJECTIVES

The NSD loss function is defined as  $\mathcal{L} = \alpha \cdot D_{\text{KL}}(\pi_{\text{ref}} \parallel \pi_{\theta}) + \mathcal{L}_{\text{GU}}$ . Specifically, we focus on isolating and analyzing the  $\mathcal{L}_{\text{GU}}$  term, which penalizes the student model on tokens vulnerable to negative conditions. Let  $\pi_c = \pi_{\theta}(c)$  denote the student’s predicted probability for the target sampled token, and  $G = \max(0, \pi_{\text{neg}} - \pi_{\text{ref}})$  serve as the adaptive gate. To demonstrate the necessity of our  $\mathcal{L}_{\text{GU}}$  item with Sigmoid design, we compare two distinct candidates for this penalty:

$$\text{Standard Unlikelihood: } \text{GU}_{\text{std}} = G \cdot [-\log(1 - \pi_c)] \quad (8)$$

$$\text{Ours: } \text{GU}_{\text{sig}} = G \cdot \sigma(-\log(1 - \pi_c)) = G \cdot \frac{1}{2 - \pi_c} \quad (9)$$

The parameter update magnitude is driven by the gradient of the loss with respect to the pre-softmax logit,  $\frac{\partial \text{GU}}{\partial z_c}$ . Given the logit-probability Jacobian  $\frac{\partial \pi_c}{\partial z_c} = \pi_c(1 - \pi_c)$ , we evaluate the optimization behavior of both formulations below.

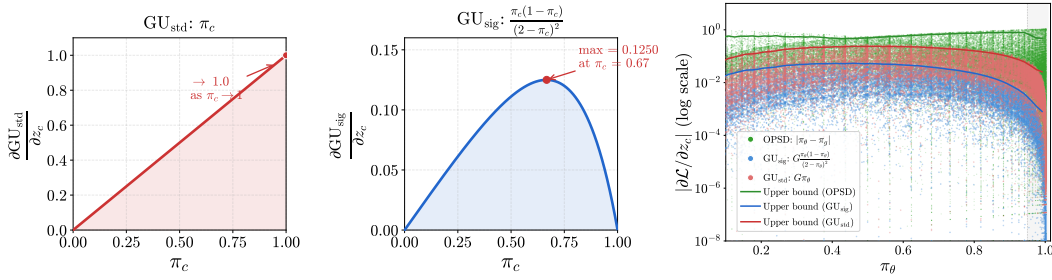


Figure 7: (Left and Mid) Comparison of the variations of two types of gradients by probability. (Right) The real gradient distribution in 100 training samples. OPSD tends to assign larger gradients to high-probability tokens, making the model more prone to drastic updates. In contrast, compared to the vanilla unlikelihood loss, our GU objective further suppresses the gradients on high-probability tokens.

### A.1 GRADIENT HAZARD IN STANDARD UNLIKELYHOOD

Applying the chain rule to the standard unbounded logarithmic penalty (Equation 8), the gradient with respect to the logit is:

$$\frac{\partial \text{GU}_{\text{std}}}{\partial z_c} = G \cdot \frac{1}{1 - \pi_c} \cdot \pi_c(1 - \pi_c) = G \cdot \pi_c \quad (10)$$

Mathematically, the gradient in standard unlikelihood scales strictly linearly with the student’s confidence  $\pi_c$  (as shown in the Figure 7). This creates an optimization hazard: In causal language modeling, tokens with extreme confidence ( $\pi_c > 0.9$ ) are probably trivial structural tokens—such as fixed collocations, prepositions, and punctuation. Under this formulation, whenever the gate  $G$  is triggered when  $\pi_c \rightarrow 1$ , the optimizer delivers its almost maximum update magnitude to these hyper-confident function words. This aggressively penalizes the model’s fundamental linguistic priors, leading to a degradation in generation fluency, especially when the high-probability token ratio is high per rollout.

### A.2 IMPLICIT GRADIENT ATTENUATION IN SIGMOID-SQUASHED GU

To construct a noise-resilient supervision signal, our method utilizes the Sigmoid-squashed penalty (Equation 9). Deriving the logit gradient for this formulation yields:

$$\begin{aligned} \frac{\partial \text{GU}_{\text{sig}}}{\partial z_c} &= G \cdot \frac{1}{(2 - \pi_c)^2} \cdot \pi_c(1 - \pi_c) \\ &= G \cdot \frac{\pi_c(1 - \pi_c)}{(2 - \pi_c)^2} \end{aligned} \quad (11)$$

This formulation introduces an elegant, parameter-free implicit gradient attenuation mechanism. The presence of the  $(1 - \pi_c)$  term in the numerator fundamentally alters the gradient landscape. As the student model’s probability approaches 1, the gradient magnitude decays toward zero:

$$\lim_{\pi_c \rightarrow 1} \frac{\partial \text{GU}_{\text{sig}}}{\partial z_c} = 0 \quad (12)$$

Since  $\pi_c > 0.9$  predominantly corresponds to uninformative syntactic tokens, the Sigmoid function inherently protects the model’s structural fluency by silencing huge gradient on these tokens. Instead, as shown in Figure 7 it naturally concentrates the highest gradient magnitude on mid-confidence tokens ( $\pi_c \approx 0.6$ ), which are more likely to be the ambiguous, reasoning-critical tokens where the student model requires the strongest corrective supervision. Consequently, our squashed formulation guarantees that dense supervision remains targeted and stable.

### A.3 OPSD GRADIENT ANALYSIS

OPSD objective can be described by:

$$\mathcal{L}_{\text{OPSD}}^{(t)} = D_{\text{KL}}\left(\pi_{\theta}(\cdot \mid x_i, s_i, y_{<t}) \parallel \pi_{\theta}(\cdot \mid x_i, y_{<t})\right) \quad (13)$$

$s_i$  denotes the gold solution in  $i$ -th training sample. The gradient visualization is shown in Figure 7. This figure shows that, compared with the NSD objective, whose gradient generally decreases as the token probability increases, the OPSD objective exhibits an increasing trend. This indicates that OPSD encourages the model to learn more from high-probability tokens, which may lead to certain forms of reward hacking, such as overlearning style tokens. In contrast, the candidate objectives in NSD exhibit relatively stable gradient patterns, while the sigmoid-based GU can more effectively suppress gradients on high-probability tokens.

## B COMPARISON OF NSD WITH OTHER METHODS

Table 4: Comparison of NSD with other methods. We conceptually compare them across the following dimensions: **Sampling** denotes whether the training relies on trajectories generated by the model itself; **Source of reward signal** indicates the core component driving the training loss function; **Teacher** specifies whether the approach depends on an external teacher model; **Gold label** refers to whether ground-truth answers are required; and **Monitor signal quality** represents whether the method actively filters training signals (e.g., unconsciously or intentionally) rather than indiscriminately optimizing over all tokens. Note that our NSD is a label-free approach, which is not directly comparable to baselines that rely on additional or external supervision. Consequently, our main experiments mostly focus on comparable methods, with OPSD as a representative label-dependent method for reference.

| Method                      | Sampling     | Source of reward signal | Teacher    | Gold label | Monitor signal quality             |
|-----------------------------|--------------|-------------------------|------------|------------|------------------------------------|
| SFT/Off-Policy Distillation | ⊗ off-policy | external teacher        | ⊗ external | ⊙ no       | ⊗ no                               |
| RLVR (GRPO)                 | ⊙ on-policy  | gold label              | ⊙ no       | ⊗ needed   | ⊙ noise gradients are counteracted |
| OPD                         | ⊙ on-policy  | external reward         | ⊗ external | ⊙ no       | ⊗ no                               |
| OPSD/SDPO                   | ⊙ on-policy  | gold label              | ⊙ self     | ⊗ needed   | ⊗ no                               |
| SD-Zero                     | ⊙ on-policy  | trained reviser         | ⊙ self     | ⊗ needed   | ⊗ no                               |
| RLIF                        | ⊙ on-policy  | internal metric         | ⊙ no       | ⊙ no       | ⊗ no                               |
| TTRL/U-OPSD                 | ⊙ on-policy  | majority-voting         | ⊙ no       | ⊙ no       | ⊗ no                               |
| <b>NSD</b>                  | ⊙ on-policy  | negative condition      | ⊙ self     | ⊙ no       | ⊙ noise is filtered by gating      |

## C EXPERIMENT DETAILS

### C.1 HYPERPARAMETERS

Table 5 lists the training hyperparameters for all methods. All experiments are conducted on a single node equipped with 8 NVIDIA A100 (80GB) GPUs. Unless otherwise specified, we adopt the default hyperparameters from the respective official implementations, with the following controlled adjustments for fair comparison: For OPSD, we evaluate configurations both with and without LoRA and report the best-performing variant (where LoRA achieves superior results on the 1.7B and 4B models). For Intuitior, we standardize the training batch size to 128, deviating from their scale-dependent defaults (64 for smaller models and 128 for larger models). For TTRL, as majority voting relies on complete final solutions, we extend the maximum generation length to 8192 tokens to prevent output truncation.

Table 5: Training hyperparameters for all methods. “—” means not applicable.

| Hyperparameter         | NSD (Online, Solution-aware) | OPSD                        | Intuitior          | TTRL               |
|------------------------|------------------------------|-----------------------------|--------------------|--------------------|
| GPUs                   | 4 for actor + 2 for teacher  | 4 for actor + 2 for teacher | 8                  | 8                  |
| Train batch size       | 32                           | 32                          | 128                | 8                  |
| PPO mini-batch size    | 32                           | 32                          | 128                | 1                  |
| Max prompt length      | 512                          | 512                         | 512                | 512                |
| Max response length    | 4096                         | 4096                        | 3072               | 8192               |
| Actor learning rate    | $1 \times 10^{-6}$           | $5 \times 10^{-6}$          | $3 \times 10^{-6}$ | $5 \times 10^{-7}$ |
| LR warmup ratio        | 0.1                          | 0.1                         | 0.1                | 0.03               |
| Rollout per sample $n$ | 1                            | 1                           | 8                  | 8                  |
| Top- $k$ logits        | 32                           | -1                          | —                  | —                  |
| KL coefficient         | 0.01                         | —                           | 0.005              | 0.00               |
| Total epochs           | 2                            | 2                           | 2                  | 2                  |

*OPSD LoRA target modules: all-linear, with LoRA rank = 64 and alpha = 128.*

### C.2 TEMPLATES

We use the Qwen3 instruct chat template throughout. All training are conducted in **non-thinking mode**: the chat template is invoked with `enable_thinking=False`, which causes the model to emit an empty `<think>` block and proceed directly to the answer. This applies uniformly to the student rollout, the teacher log-probability computation, and all downstream evaluations.

The template for a single-turn exchange takes the following form:

Qwen3 Chat Template (non-thinking, enable\_thinking=False)

```
<|im_start|>user
{user message}
<|im_end|>
<|im_start|>assistant
<think>

</think>

{model response}
```

The empty `<think>...</think>` block is prepended automatically by the template when `enable_thinking=False` and `add_generation_prompt=True`. The model then generates its response after the second blank line.

### C.3 PROMPTS

The student always receives the plain problem prompt below. During NSD training the teacher receives either the same prompt (reference pass) or a negative prompt (negative pass), depending on the variant. All prompts are wrapped in the chat template described in Appendix C.2.

**Student / reference teacher prompt (all methods).****Student Prompt**

Problem: {problem}

Let's think step by step and output the final answer within \boxed{ }.

**NSD negative condition prompt generator (Question-only).** The following meta-prompt is sent to a helper LLM to produce the per-sample negative condition prompt  $n_i$  used in the question-only offline variant. The generated prompt replaces the system context seen by the teacher model.

**Meta-Prompt: Question-only Negative Condition Generation**

You are an expert Math Educator and AI Prompt Engineer. Your task is to analyze the following math problem and generate a "Generalized Attack Prompt" that will force an LLM to make a highly plausible, human-like cognitive error.

**Anatomy of a Universal Attack Prompt:**

1. **Persona:** Must start exactly with "*You are a student who...*". Describe a specific bad habit relevant to *this* problem.
2. **Trigger:** Abstract the problem's mathematical class. *Never* use specific numbers or variables from the current problem.
3. **Flawed Execution:** Instruct a naive heuristic or impulsive shortcut that would give a wrong answer.
4. **Fatal Omission:** Explicitly forbid the critical verification step.

Now, perform this task for the following problem:

**Problem:** {problem}

Output *only* the "Generalized Attack Prompt". Start your response with "*You are a student who...*". Keep it concise (2–3 sentences).

**NSD negative condition prompt generator (Solution-aware).** When the model's own rollout is available, the meta-prompt is augmented with the student's solution to produce a more targeted negative condition.

**Meta-Prompt: Solution-aware Negative Condition Generation**

You are an expert Math Educator and AI Prompt Engineer. Your task is to analyze the following math problem and a student's existing solution, then generate a "Targeted Attack Prompt" that exploits the exact reasoning steps the student used to cause a highly plausible cognitive error.

The student's solution reveals *how* they solved this problem—use that to craft an attack targeting their specific reasoning steps.

**Anatomy of a Targeted Attack Prompt:**

1. **Persona:** Must start exactly with "*You are a student who...*". Describe a specific bad habit that would corrupt the *exact* step where this student's reasoning is most fragile.
2. **Trigger:** Reference the *type* of reasoning the student used (not specific numbers or variables from this problem).
3. **Flawed Execution:** Instruct a shortcut that mirrors the student's approach but introduces a subtle error.
4. **Fatal Omission:** Forbid the specific verification the student performed correctly.

**Problem:** {problem}

**Student's Existing Solution:**  
{solution}

Output *only* the "Targeted Attack Prompt". Start your response with "*You are a student who...*". Keep it concise (2–3 sentences).

**NSD teacher prompt (wiki-irr variant).** In the wiki-irr variant no meta-prompt generator is used. Instead, each training sample is paired with a randomly sampled Wikipedia passage that is

concatenated as spurious “context”. The teacher sees the following prompt while the student still receives the plain student prompt above.

**Teacher Prompt:** Wiki Irrelevant Negative Condition

Problem: {problem}  
 Below is some context you may find useful to answering the question above:  
 {wikipedia\_passage}  
 Let’s think step by step and output the final answer within \boxed{}

**NSD teacher prompt.** For the question-only and solution-aware offline variants, the teacher receives the following prompt, where {negative\_condition} is the output of the meta-prompt generator above.

**Teacher Prompt:** Question-only / Solution-aware Negative Condition

Problem: {problem}  
 {negative\_condition}  
 Now solve the problem following this instruction:  
 Let’s think step by step and output the final answer within \boxed{}

## D ADDITIONAL EXPERIMENTAL RESULTS

### D.1 PASS@8 PERFORMANCE

We report the performance of pass@8 in Table 6.

Table 6: Main evaluation results reported as pass@8 (%): at least one of 8 sampled solutions is correct. Same evaluation setting as Table 1.  $\Delta$  Avg is the average absolute improvement over the same-size baseline across all 7 benchmarks. **Bold** marks the best result in each model-size group; underline marks the second best.  $\dagger$  denotes methods that require ground-truth labels.

| Method             | AIME<br>2024 | AIME<br>2025 | AIME<br>2026 | HMMT<br>2025 Feb | AMC<br>2023 | Olympiad-<br>Bench | MATH-<br>500 | $\Delta$ Avg |
|--------------------|--------------|--------------|--------------|------------------|-------------|--------------------|--------------|--------------|
| <i>1.7B Models</i> |              |              |              |                  |             |                    |              |              |
| Qwen3-1.7B         | 16.7         | 23.3         | 13.3         | 16.7             | 70.0        | 57.8               | 77.6         | —            |
| OPSD $\dagger$     | <b>40.0</b>  | 23.3         | 13.3         | 16.7             | 72.5        | <u>59.6</u>        | 77.6         | +3.9         |
| Intuitior          | 30.0         | 23.3         | 16.7         | 13.3             | 75.0        | <u>57.0</u>        | 76.2         | +2.3         |
| TTRL               | 30.0         | <u>26.7</u>  | <u>23.3</u>  | 13.3             | <u>77.5</u> | 58.7               | 77.0         | +4.4         |
| <b>NSD</b>         | <u>33.3</u>  | <b>36.7</b>  | <b>23.3</b>  | <b>16.7</b>      | <b>77.5</b> | <b>60.9</b>        | <b>77.6</b>  | <b>+7.2</b>  |
| <i>4B Models</i>   |              |              |              |                  |             |                    |              |              |
| Qwen3-4B           | 50.0         | 40.0         | 40.0         | 20.0             | 95.0        | 67.0               | 81.6         | —            |
| OPSD $\dagger$     | 40.0         | 46.7         | 36.7         | <u>30.0</u>      | 92.5        | 66.4               | 81.6         | +0.0         |
| Intuitior          | 50.0         | <u>53.3</u>  | 36.7         | 26.7             | <u>95.0</u> | 66.1               | 80.0         | +2.0         |
| TTRL               | <u>63.3</u>  | 40.0         | <u>46.7</u>  | 23.3             | 90.0        | 64.3               | 81.6         | +2.2         |
| <b>NSD</b>         | <b>60.0</b>  | <b>63.3</b>  | <b>53.3</b>  | <b>30.0</b>      | <b>95.0</b> | <b>68.3</b>        | <b>82.0</b>  | <b>+8.3</b>  |
| <i>8B Models</i>   |              |              |              |                  |             |                    |              |              |
| Qwen3-8B           | 56.7         | 30.0         | 43.3         | 23.3             | 92.5        | 66.8               | 82.0         | —            |
| OPSD $\dagger$     | 46.7         | <u>43.3</u>  | 40.0         | 23.3             | 87.5        | 67.6               | 81.8         | −0.6         |
| Intuitior          | <u>60.0</u>  | 40.0         | 36.7         | <u>26.7</u>      | <u>95.0</u> | <u>69.0</u>        | <u>81.8</u>  | +2.1         |
| TTRL               | 56.7         | 33.3         | 36.7         | 20.0             | 92.5        | 66.8               | 82.0         | −1.0         |
| <b>NSD</b>         | <b>70.0</b>  | <b>46.7</b>  | <b>60.0</b>  | <b>40.0</b>      | <b>95.0</b> | <b>69.0</b>        | <b>81.8</b>  | <b>+9.7</b>  |



## D.2 PERFORMANCE ON THINKING MODE

We evaluate the performance of all methods under the thinking mode on the Qwen3-4B model, reporting the results of the best-performing checkpoints evaluated under the thinking mode. As shown in Table 7, NSD also outperforms all other baselines overall. Notably, both OPSD and NSD achieve more improvements on challenging datasets such as AIME and Olympiad Bench, aligning with the observations from the non-thinking setting. On datasets with limited headroom for improvement (e.g., AMC), all methods perform comparably to the base model. Furthermore, we observe that the Intuitior-trained model tends to over-think, causing many responses to exceed the maximum generation length limit (even after extending it to 38k tokens), which leads to a severe degradation in accuracy. This phenomenon is also discussed in the previous works (Zhao et al., 2026b; Ghimire et al., 2026).

Table 7: Thinking mode evaluation results on 4B models reported as avg@8 (%). Same evaluation setting as Table 1.

| Model     | AIME<br>2024 | AIME<br>2025 | AIME<br>2026 | HMMT<br>2025 | AMC<br>2023 | MATH-<br>500 | Olympiad<br>Bench | $\Delta$<br>Avg |
|-----------|--------------|--------------|--------------|--------------|-------------|--------------|-------------------|-----------------|
| Qwen3-4B  | 75.8         | 69.1         | 67.5         | 46.0         | 97.2        | 79.8         | 45.9              | -               |
| OPSD      | 76.2         | 69.8         | 67.2         | 46.2         | 96.6        | <b>80.0</b>  | 46.7              | +0.2            |
| Intuitior | 52.9         | 45.8         | 51.2         | 39.6         | 90.9        | 78.1         | 43.9              | -11.3           |
| TTRL      | 72.5         | 64.3         | 65.1         | 46.0         | 96.6        | 79.2         | 44.4              | -1.9            |
| NSD       | <b>77.3</b>  | <b>73.3</b>  | <b>67.7</b>  | <b>48.4</b>  | <b>97.8</b> | <u>79.9</u>  | <b>57.9</b>       | <b>+3.0</b>     |

## D.3 ALTERNATIVE OBJECTIVE: POLICY GRADIENT OPTIMIZATION

While the NSD loss  $\mathcal{L}_{\text{NSD}}^{(t)}$  can be directly backpropagated as a supervised objective, we find it also fits a sampled-token advantage policy-gradient framework, following the spirit of Zhao et al. (2026a) and Lu & Lab (2025). For each token  $y_t$  in a student rollout  $y \sim \pi_\theta(\cdot | x_i)$ , we define a token-level advantage as the NSD loss:

$$A_t = -\mathcal{L}_{\text{NSD}}^{(t)} \quad (14)$$

Intuitively, a token with high NSD loss receives a strongly negative advantage, signaling the policy to reduce its probability. Conversely, tokens with low NSD loss receive near-zero or positive advantage, leaving their probabilities unchanged. We treat  $A_t$  as a constant with respect to  $\theta$  and optimize the student via the standard policy gradient surrogate objective:

$$\mathcal{J}_{\text{PG}}(\theta) = \mathbb{E}_{(x,a) \sim \mathcal{D}, y \sim \pi_\theta} \left[ \sum_t A_t \log \pi_\theta(y_t | x_i, y_{<t}) \right] \quad (15)$$

We evaluate the NSD based on the alternative objective under the same setting as our main experiment. The result is shown in Table 8. Compared to models optimized with the  $\mathcal{J}$  objective (Eq. 6), NSD trained under  $\mathcal{J}_{\text{PG}}$  (Eq. 15) achieves superior performance on the 1.7B model (+5.0% on average). However, on the 4B and 8B models, the  $\mathcal{J}$ -objective NSD yields better overall results. Notably, the wiki-irr strategy consistently performs best under the policy gradient setting, while the online solution-aware strategy emerges as the second best. This discrepancy arises because the gradients are truncated by the advantage function, decreasing the capture of richer gradient signals. In contrast, wiki-irr utilizes noise to introduce more generalized interference (causing an overall degradation of the model’s reasoning capabilities in long contexts), which ultimately makes it a more effective strategy in this regime.

## D.4 CASE STUDY

A case study is shown in Table 9. These results indicate that NSD-trained models more readily explore novel and correct solutions that are entirely absent from the outputs of both the base and OPSD-trained models. Furthermore, by prompting an external LLM (Sonnet) to analyze the reasoning traces of each response, we observe that the NSD-trained model engages in several reflection steps, successfully circumventing erroneous trajectories that commonly trap the base model.

Table 8: Evaluation results of NSD with different negative conditioning strategies on mathematical reasoning benchmarks. The models are trained based on the NSD policy gradient objective in Eq. 15.

| Method / Variant               | AIME 2024   | AIME 2025   | HMMT Feb 2025 | MATH-500    | $\Delta$ Avg |
|--------------------------------|-------------|-------------|---------------|-------------|--------------|
| <i>1.7B Models</i>             |             |             |               |             |              |
| NSD (Solution-aware, Offline)  | 15.8        | 14.2        | 5.4           | 63.2        | +2.4         |
| NSD (Solution-aware, Online)   | 15.8        | 12.5        | 7.5           | 63.3        | +2.5         |
| <b>NSD (Wiki-irr, Offline)</b> | <b>20.0</b> | <b>15.0</b> | <b>9.6</b>    | <b>64.4</b> | <b>+5.0</b>  |
| <i>4B Models</i>               |             |             |               |             |              |
| NSD (Question-only, Offline)   | 30.4        | 24.6        | 16.2          | 72.7        | +4.4         |
| NSD (Solution-aware, Offline)  | <b>33.8</b> | 22.5        | 14.6          | 72.1        | +4.2         |
| NSD (Solution-aware, Online)   | 31.3        | 25.0        | 16.3          | <b>74.0</b> | <b>+5.1</b>  |
| NSD (Wiki-irr, Offline)        | <u>33.8</u> | <b>25.4</b> | <b>17.1</b>   | <u>73.5</u> | <b>+5.9</b>  |
| <i>8B Models</i>               |             |             |               |             |              |
| NSD (Solution-aware, Offline)  | 30.8        | 23.3        | 12.5          | 73.6        | +1.9         |
| NSD (Solution-aware, Online)   | <b>35.0</b> | 21.7        | 13.8          | 73.6        | +2.8         |
| <b>NSD (Wiki-irr, Offline)</b> | <u>34.2</u> | <b>28.8</b> | <b>19.2</b>   | <u>73.9</u> | <b>+5.8</b>  |

Table 9: Case study on AIME 2025 II #12 comparing Qwen3-4B baseline, OPSD, and NSD. The table shows a summary from Sonnet. Numbers denote correct samples out of 8 independent draws. ✓ correct; ✗ incorrect.

| Problem                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | Baseline                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | OPSD                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | NSD (Ours)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><i>AIME 2025 II #12 — Geometry.</i> Let <math>A_1 A_2 \cdots A_{11}</math> be a non-convex simple 11-gon satisfying: (1) <math>[A_i A_1 A_{i+1}] = 1</math> for <math>2 \leq i \leq 10</math>; (2) <math>\cos(\angle A_i A_1 A_{i+1}) = \frac{12}{13}</math> for <math>2 \leq i \leq 10</math>; (3) perimeter = 20. Express <math>A_1 A_2 + A_1 A_{11} = \frac{m\sqrt{n-p}}{q}</math> (<math>n</math> squarefree, no prime divides all of <math>m, p, q</math>); find <math>m + n + p + q</math>.</p> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Pass@8</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | 0/8 ✗                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | 0/8 ✗                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | 4/8 ✓                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Key reasoning</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | <p>From <math>\cos \theta = \frac{12}{13}</math> derives <math>\sin \theta = \frac{5}{13}</math>, hence <math> A_1 A_i  \cdot  A_1 A_{i+1}  = \frac{26}{5}</math>. The product constraint gives an alternating sequence <math>a_2 = x, a_3 = \frac{26}{5x}, a_4 = x, \dots</math>. Attempts to use the perimeter but conflates the sum of radii from <math>A_1</math> with the polygon perimeter, obtaining <math>5x + \frac{26}{x} = 20</math> which has no clean closed form. After extensive numerical trials, <b>guesses the symmetric solution</b> <math>x = \frac{13}{\sqrt{5}}</math> (i.e. <math>a_2 = a_{10}</math>), giving <math>A_1 A_2 + A_1 A_{11} = \frac{13}{\sqrt{5}} + 2\sqrt{5} = \frac{23\sqrt{5}}{5}</math>, so <math>m = 23, n = 5, p = 0, q = 5</math>. <b>Final: 33 ✗</b></p> | <p>Same product relation and alternating sequence. Applies Law of Cosines: since <math>x_i x_{i+1} = \frac{26}{5}</math>, each inner-polygon side satisfies <math>d^2 = x_i^2 + x_{i+1}^2 - \frac{48}{5}</math>, so all 9 inner sides are equal. Correctly writes the perimeter equation <math>a + 9d + \frac{26}{5a} = 20</math> and sets <math>S = a + \frac{26}{5a}</math>. <i>Instead of solving for <math>S</math>, minimises <math>S</math> via AM-GM:</i> <math>\min(a + \frac{26}{5a}) = 2\sqrt{\frac{26}{5}} = \frac{2\sqrt{130}}{5}</math>, and incorrectly <b>treats this minimum as the answer</b>, concluding <math>A_1 A_2 + A_1 A_{11} = \frac{2\sqrt{130}}{5}</math>, so <math>m = 2, n = 130, p = 0, q = 5</math>. <b>Final: 137 ✗</b></p> | <p>Same product relation and alternating sequence. Applies Law of Cosines; all 9 inner sides equal <math>d</math>. Writes the perimeter equation <math>a + 9d + \frac{26}{5a} = 20</math>. Then <b>attempts a symmetric-guess approach</b>: tests <math>x = \sqrt{26/5}</math>, <math>x = 2</math>, <math>x = 13/5</math>, <math>x = 13/\sqrt{5}</math> in turn, each time verifying numerically that the two expressions for <math>d^2</math> do not agree. Sets <math>S = a + \frac{26}{5a}</math>, so the perimeter equation gives <math>d = \frac{20-S}{9}</math>. Rewrites <math>d^2</math> via Law of Cosines: <math>d^2 = a^2 + \frac{676}{25a^2} - \frac{48}{5} = (a + \frac{26}{5a})^2 - \frac{52}{5} - \frac{48}{5} = S^2 - 20</math>. Substituting <math>d = \frac{20-S}{9}</math> yields <math>(\frac{20-S}{9})^2 = S^2 - 20</math>, which expands to <math>4S^2 + 2S - 101 = 0</math>. Quadratic formula: <math>S = \frac{-2 \pm \sqrt{1620}}{8} = \frac{9\sqrt{5}-1}{4}</math> (positive root), so <math>m = 9, n = 5, p = 1, q = 4</math>. <b>Final: 19 ✓</b></p> |
| <b>Reflection</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | <p><i>No self-correction.</i> After the perimeter approach yields no clean form, the model <b>commits to a guess</b> (<math>x = \frac{13}{\sqrt{5}}</math>) without checking whether it satisfies the original constraints, and submits the result directly.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | <p><i>No self-correction.</i> The model sets up the correct equation structure but <b>replaces the constraint with its relaxation</b>: once the AM-GM bound is computed it is treated as the solution, with no attempt to verify that the minimum is actually attained.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | <p>After the guessing strategy fails on multiple candidates (<math>x = \sqrt{26/5}</math>, <math>2, \frac{13}{5}, \frac{13}{\sqrt{5}}</math>), the model <b>explicitly abandons the approach</b> (“<i>Hmm. Maybe my approach is not working. Alternative idea:</i>”) and <b>reframes the problem</b> around the aggregate variable <math>S = a + \frac{26}{5a}</math>, turning an intractable system into a single quadratic <math>4S^2 + 2S - 101 = 0</math>.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |

## E DEFINITION OF TASK, STYLE, AND REFLECTION TOKENS

Considering the similar task setting with RLCSO (Pan et al., 2026), we follow their definition of both task and style tokens:

- (1) empty or whitespace-only  $\rightarrow$  **style**;
- (2) matches any of the math regexes (a digit  $\backslash d$ ; an arithmetic operator in  $+ - = */ < >$   $\times \div \leq \geq \neq$ ; a LaTeX command  $\backslash [A-Z a-z]^+$ ; a double backslash; or one of  $\$ \wedge \_$ )  $\rightarrow$  **task**;
- (3) the normalized form is in the math wordlist {mod, prime, factor, gcd, lcm, log, ln, sin, cos, tan, exp, integral, sqrt, boxed, frac, sum, prod, pi, alpha, beta, gamma, theta, delta, lambda, mu, sigma, infinity, leq, geq, neq, cdot, times, div}  $\rightarrow$  **task**;
- (4) pure punctuation or a literal newline token ( $\backslash n, \backslash \backslash n$ )  $\rightarrow$  **style**;
- (5) the normalized form is in the discourse wordlist (connectives therefore, so, thus, hence, then, because, since; hedges wait, maybe, perhaps, seems, okay, ok, well, now, first, next, finally, actually, alternatively, however; scaffolding step, answer, let, lets; closed-class function words is, are, us, we, the, a, an, of, to, for, in, on, by, at, as, and, or, but, if, yes, no, this, that, these, those, it, its, be, been, being, have, has, had, do, does, did, will, would, should, could, can, may)  $\rightarrow$  **style**;
- (6) otherwise  $\rightarrow$  **neutral**.

The following tokens are considered as reflection tokens:

wait, actually, hmm, let me reconsider, let me rethink, i made an error, i made a mistake, that's wrong, that is wrong, incorrect, reconsider, rethink, re-examine, let me check, let me verify, double check, double-check, going back, revisit, on second thought